

Finding the problem: observation, hypothesis, code, numbers

1. Observation, from reading the original system's failures

Held-out question: "דרכון דחוף" (urgent passport). Correct page: הוצאת דרכון זמני (temporary passport).

Returned: 1. biometric passport 2. temporary passport

The pattern repeats: right topic, wrong page, and the wrong page is a sibling of the right one (couple with / without children; income top-up for old-age / survivors' pension).

2. Hypothesis

Sibling pages have near-identical bodies; what separates them is the page title. If the model never sees the title, it has nothing to tell them apart with.

3. Check in the code

doc-config.json: field_to_embed = "content". The model receives that field only; the title is stored next to the paragraph but never embedded.

4. Confirm with numbers (296 questions of the training code's validation split, 24,487 paragraphs)

Original system: rank 1 for 36.8%, top 3 for 58.1%. The Demo sends 3 pages to the LLM, so for 42% of questions the right one is not among them.

187 misses by rank of the right page:

- rank 2–3: 63, mostly a sibling page first. The pattern above.
- rank 4–10: 53, mixed; generic guide pages win.
- beyond 10: 71, questions that do not identify a page on their own. The ceiling.

64% of paragraphs already open with their section heading, so the section is in the vector. The only missing piece is the page title.

The fix: embed the page title with the paragraph, tested on three sets

What changed

One key in doc-config.json: `embed_context_fields: ["title"]`. The text embedded at indexing time becomes "`<page title>\n<paragraph>`". The model is not retrained. Query, search, index layout, API and frontend are unchanged. Remove the key and re-index for the original.

Variant	Webiks, 296	Clean, 150	Public, 300
Original system	36.8 / 58.1	53.3 / 74.7	55.3 / 74.7
Title in text (shipped)	46.3 / 64.5	60.0 / 78.7	53.7 / 75.7
+ title vector 0.3 (tested, off)	52.7 / 70.9	56.0 / 77.3	54.3 / 73.0
+ LLM title rerank (optional, off)	60.1 / 75.3	66.7 / 84.0	60.7 / 77.3

rank 1 / top 3, %. The rerank row comes from `title_llm.py`, whose own baselines are within 2 questions of these.

Three sets

- Webiks: their training code's validation split. The file was training data, so the claim is the difference, not the level.
- Clean: 150 pages with no question in that file; one question each, written from the body with the title hidden.
- Public: the 300 questions a competitor published in his fork.

Why the title vector was switched off

Shipped until 04.10. +6.4pp on the Webiks set, but both independent sets gave it back, it got worse with every increase in weight, and it hurt significantly where the title does not describe the body. Its weight was chosen on questions written with the page in view. It stays in the engine, off.

Optional second stage, off by default, and what was tested and rejected

LLM title rerank

- Search returns 30 candidate pages; a chat model sees the question and the 30 titles and returns the best ones in order.
- LLM factory: OpenAI and any OpenAI-compatible server (Azure, Ollama, vLLM), Anthropic, Gemini, or any model registered in a few lines.
- Turn on: `TITLE_RERANK_ENABLED=true` in `app/.env`, provider and model of choice.
- On any failure (key, timeout, bad reply) it falls back to the plain retrieval order.

Cost: ~0.8 s per query, an LLM key, and no effect in mock mode. Hence off. 19 tests; 60 pass overall.

Experiment	Δ rank 1 / top 3, pp	Why not
256-token chunks + title	-5.8 / +2.7	vs title in text: trades rank 1 for top 3, doubles the index, LLM gets half-paragraphs
LLM-written context per paragraph	-3.7 / -0.6	vs the same variant without it; cost \$3.34
Rewrite / HyDE / 4 paraphrases	up to +2.4 / +3.8	heavy churn both ways; an LLM in the search path
Boilerplate filter / 500 candidates	± 1.5 / 0	a page ranks by its best paragraph

296 Webiks questions; each against the same setup without it

Next

Test the rerank on real traffic and against a paragraph-level cross-encoder · small-to-big retrieval · a human-written question set · fix corpus pages whose text belongs to another page (up to 271 suspects).